

CO543 / CO5430
Computer Vision Project - 2026
Vehicle or Pedestrian Tracking in Video
Report

Group Number	G07
Project ID	P07
Project Category	Object Tracking
Dataset Source	MOT16 - https://www.kaggle.com/datasets/takshmandar/mot16-dataset
GitHub Repository	https://github.com/cepdnaclk/e23-co5430-vehicle-pedestrian-tracking

Group Members

#	Name	Registration No.
1	M.R.A. Rahman	E/23/282
2	A. Piraveen	E/23/273
3	M.M.M.A. Nular	E/23/249
4	Thanush V.	E/23/392

Abstract

Tracking every pedestrian in a busy street camera feed, and keeping each person's identity straight even when they disappear behind a pillar or a crowd, is one of the harder everyday problems in computer vision. This report walks through how our group tackled it on the MOT16 benchmark. We fine tuned a YOLOv8n detector to find pedestrians, then built two different ways of linking those detections into tracks across frames: a SORT tracker we wrote ourselves from the ground up, and ByteTrack, which comes built into the Ultralytics framework we were already using. On a held out slice of the data the detector reached a mean average precision of 0.9058 at fifty percent overlap, with a precision of 0.9237 and a recall of 0.8308. When we compared the two trackers on the MOT16-02 sequence, SORT came out ahead on raw accuracy, MOTA 0.684 against 0.655, but ByteTrack cut identity switches from 98 down to 40, a fifty nine percent drop, which was really the whole point of adding it. The pages that follow explain where the data came from, how we prepared it, how both trackers actually work under the hood, what we found, where things still go wrong, and what we would do next given more time.

1. Introduction

Picture a CCTV camera watching a busy plaza. Thirty times a second it captures a frame, and somewhere in that stream of images are dozens of people walking, stopping, crossing paths, and occasionally vanishing behind each other for a moment. The task this project takes on, multiple object tracking, is to find every one of those people and give each of them a number that stays the same for as long as they remain in view, even through a brief occlusion. It sounds simple to state but is genuinely hard to do well, because people change scale as they move toward or away from the camera, overlap constantly in crowded scenes, and the camera itself is not always static.

Our approach splits the problem into two stages that are trained and reasoned about separately. First, a detector looks at each frame on its own and draws a box around every pedestrian it finds, with no notion of time or identity. Second, a tracking algorithm takes those per frame boxes and stitches them into trajectories, deciding frame by frame which new box belongs to which existing person. We built this pipeline twice over for the second stage, once with a classic motion only tracker, SORT, and once with a newer alternative, ByteTrack, specifically so we could compare them head to head rather than just picking one on faith. The rest of this report follows the project in the order it was actually built: the data, the preprocessing, the two trackers, the numbers we measured, and what we would still like to fix.

2. Related Work

The tracker we implemented from scratch is based on SORT, introduced by Bewley et al., which pairs a Kalman filter for predicting where each person should be next with the Hungarian algorithm for deciding which prediction matches which new detection. It is deliberately simple, has no idea what a person looks like beyond their box, and relies entirely on motion. That simplicity is exactly why we chose it as our baseline.

Wojke, Bewley, and Paulus later extended this idea into DeepSORT by adding a learned appearance feature, letting the tracker recognise a person by how they look and not just where they were a moment ago, which helps considerably after an occlusion. We did not end up implementing DeepSORT ourselves in the time available, and note it here as work we would still like to add. Instead, our second tracker is ByteTrack, from Zhang et al., which takes a different angle on the same occlusion problem: rather than adding appearance, it simply refuses to throw away low confidence detections outright, giving them a second chance to be matched once the obvious high confidence matches are settled. Because Ultralytics, the same framework we already use for YOLOv8, ships with ByteTrack built in, it was the more practical of the two upgrades to get running and evaluate properly.

For the dataset and the evaluation protocol itself we follow Milan et al., who defined the MOT16 benchmark along with the MOTA and MOTP metrics used throughout Section 6. The detector underneath everything is YOLOv8, from Jocher, Chaurasia, and Qiu at Ultralytics; we use the smallest variant, YOLOv8n, mainly because our training hardware, a shared Google Colab GPU, made the larger variants impractical within the project timeline.

3. Dataset

We used MOT16, a well known benchmark distributed through a Kaggle mirror of the original motchallenge.net release. The footage was recorded by the benchmark's creators at real public locations, and every single box in it was drawn by hired human annotators watching the video frame by frame, not by us and not by any model. That ground truth is what we trained against and what we measured our own results relative to. Seven of the fourteen sequences in MOT16 come with this ground truth attached; the table below lists the ones we used.

Sequence	Frames	Resolution	Scene
MOT16-02	600	1920 x 1080	Venice plaza, Italy
MOT16-04	1050	1920 x 1080	ETH Bahnhof, Zurich
MOT16-05	837	640 x 480	Venice pedestrian street
MOT16-09	525	1920 x 1080	ETH Sunnyday
MOT16-10	654	1920 x 1080	ETH Crossing
MOT16-11	900	1920 x 1080	PETS09 S2L1, indoor mall
MOT16-13	750	1920 x 1080	Town Centre, UK street

Table 1. The seven annotated MOT16 sequences used, 5316 frames in total.

Each annotation line in the raw ground truth file records a frame number, a person's identity, a box given by its top left corner, width and height in pixels, a confidence flag, a class label, and a visibility fraction. We kept only real annotations, confidence equal to one, only pedestrians and cars, and only boxes at least 30 percent visible, discarding the rest as too unreliable to train on. That left 79799 pedestrian boxes across 5316 frames. No car boxes survived this filtering at all, since MOT16 barely contains any to begin with, a limitation we come back to in Section 6.

YOLO needs boxes described differently to the raw format, as a normalised centre point rather than a pixel corner, so a script we wrote, preprocess.py, converts every surviving box automatically, and a second script, verify_labels.py, draws the converted boxes back onto sample frames so we could check the maths by eye before trusting it. Each sequence was split chronologically, the first 80 percent of its frames for training and the last 20 percent for validation, rather than shuffling frames randomly, since neighbouring frames in a video look almost identical and a random split would let the model effectively see the answer in advance. This gave us 4254 training frames and 1062 validation frames overall.

4. Methodology

The pipeline has two independent stages. A detector, described in Section 4.1, finds pedestrians in each frame on its own, with no memory of previous frames. A tracker, described in Sections 4.2 and 4.3, then links those detections into identities over time. We built and compared two trackers rather than one so that the improvement from ByteTrack could be measured directly against our own SORT implementation, using the exact same detector underneath both.

4.1 Detection: YOLOv8n

YOLOv8n is a compact convolutional network, around 3.2 million parameters, that we started from COCO pretrained weights and fine tuned on our 4254 training frames for 50 epochs at a batch size of 16, resizing every frame to 640 by 640 pixels. Over the course of training the box loss fell from 1.388 to 0.650 and the classification loss from 1.189 to 0.367, while mean average precision, precision, and recall all climbed steadily, which is a good sign that the model was actually learning rather than memorising. At inference we discard anything below 0.30 confidence and apply non maximum suppression at an IoU of 0.45 to remove duplicate boxes. The whole training run took about half an hour on a single Colab T4 GPU, versus an estimated 8 to 12 hours had we been stuck on a CPU, and produced a 5.9 megabyte weights file, models/best.pt.

4.2 Tracking: SORT

We wrote our SORT implementation ourselves rather than importing one. For each frame it does five things in order: run the detector to get fresh boxes with no identity attached; ask every existing track's Kalman filter where it expects that person to be now, based on its last known position and velocity; build an overlap matrix between those predictions and the fresh boxes and let the Hungarian algorithm find the best overall pairing, accepting a match only above 0.30 IoU; drop any track that has gone three frames without a match and only start displaying a new track once it has been confirmed for three consecutive frames; and finally draw the surviving tracks and log them to a results file in the standard MOTChallenge format for later evaluation. On MOT16-02 alone this produced 8825 logged detections across 600 frames, roughly 14.7 tracked people per frame on average.

4.3 Tracking: ByteTrack

The weak point of SORT is what happens when someone briefly walks behind an obstacle. Their detection confidence dips below 0.30, SORT simply has nothing to match that frame, and after three such frames the track is deleted; when the person reappears they get treated as brand new and given a fresh identity, which shows up as an identity switch in our results. ByteTrack's fix is to run the matching twice. The first pass is identical to SORT and mops up the easy cases. The second pass takes whatever tracks were left unmatched, on the assumption they might just be temporarily hidden, and tries to pair them against the low confidence detections, roughly 0.10 to 0.30, that would otherwise be thrown away outright. If the half hidden person shows up faintly in that range, their original identity survives. Since Ultralytics already ships ByteTrack alongside YOLOv8, getting it running was mostly a configuration change rather than new algorithm code, which let us focus our time on running a fair comparison instead.

5. Experimental Setup

Detection accuracy was measured on the 1062 frame validation split held out in Section 3, using Ultralytics' own validation routine to report mean average precision at IoU 0.50, mean average precision averaged from 0.50 to 0.95, precision, and recall for the pedestrian class.

For tracking, we wrote our own evaluation script, `evaluate.py`, that walks through the MOT16-02 ground truth frame by frame and compares it against a tracker's output. A predicted box that overlaps a real pedestrian by at least half and keeps the same identity it had a frame earlier counts as a true positive; a predicted box with nothing real behind it is a false positive; a real pedestrian with no predicted box is a false negative; and a match whose identity has changed since last frame is logged as an identity switch. From these counts we compute MOTA, one minus the combined error rate of false positives, false negatives, and identity switches relative to the total number of real objects, and MOTP, the average positional error of the boxes that did match correctly, where lower is tighter alignment.

6. Results

6.1 Detection Results

Metric	Value	What it Means
mAP at IoU 0.50	0.9058	How often a prediction counts as correct at 50 percent overlap.
mAP (0.50 to 0.95)	0.5915	The same idea, averaged over a stricter range of overlap thresholds.
Precision	0.9237	How many of the predicted boxes were actually right.
Recall	0.8308	How many of the real pedestrians the model actually found.

Table 2. Detection performance on the held out validation split.

These numbers are comfortably strong for a model this small. The gap between the lenient 0.50 score and the stricter averaged score is normal for a compact single stage detector and mostly reflects boxes that are in roughly the right place but not pixel perfect, something a larger model or higher input resolution would likely tighten up.

6.2 Qualitative Tracking Results

Running the full pipeline on several sequences, the Venice plaza crowd in MOT16-02, the mall walkway in MOT16-09, the indoor shopping centre in MOT16-11, and the outdoor street scene with vehicles in MOT16-13, the tracker held onto stable boxes and consistent identities for pedestrians in light to moderate crowds, visibly following the same person across dozens of frames in both the SORT and ByteTrack output videos.

6.3 SORT versus ByteTrack

Metric	SORT	ByteTrack
MOTA, higher is better	0.684	0.655
MOTP, lower is better	0.151	0.161
Identity switches	98	40
True positives	7486	7656
False positives	1338	1827
False negatives	1354	1184

Table 3. Tracking evaluation on MOT16-02, 600 frames, 8840 ground truth objects.

SORT wins on raw MOTA simply because it is the more cautious of the two and lets through fewer false positives. ByteTrack trades some of that caution for a much better memory of who is who: identity switches drop from 98 to 40, a fifty nine percent cut, which is precisely the failure mode ByteTrack was designed to fix, and it also finds slightly more real pedestrians overall. Which tracker is actually better depends on what matters more for the use case, raw box accuracy or keeping identities stable through a crowd, and for anything resembling long term tracking we would lean toward ByteTrack despite its slightly lower MOTA.

7. Limitations, Repository, and Future Work

A few gaps are worth being upfront about. No car detections survive our current filtering, so vehicle tracking is not yet functional and would need a supplementary dataset such as UA DETRAC to fix. DeepSORT, the appearance based tracker from our original proposal, was never actually built, so the identity switch problem is only partly solved by ByteTrack rather than fully explored. Our tracking comparison in Section 6.3 was also run on a single sequence, MOT16-02, so we cannot yet claim the same pattern holds everywhere in MOT16. Finally, YOLOv8n occasionally misses very small, distant pedestrians, which a larger model variant or a higher input resolution would likely help with.

Everything described in this report, including the code, the trained weights, and the output videos, is kept in our GitHub repository at <https://github.com/cepdnack/e23-co5430-vehicle-pedestrian-tracking>, with `preprocess.py` and `verify_labels.py` handling data preparation, `sort_tracker.py` and `run_tracker.py` running the SORT pipeline, `run_bytetrack.py` running the ByteTrack pipeline, `evaluate.py` computing the metrics in Section 6, and `models/best.pt` holding the final trained detector.

8. Conclusion

Over the course of this project we took MOT16 from a folder of raw annotated frames to a full working tracker, trained a pedestrian detector that performs well above what we initially targeted, and built and fairly compared two different ways of keeping track of who is who across a video. The headline result is straightforward: ByteTrack cuts identity switches nearly in half compared with our own SORT implementation, at a small cost in overall accuracy, which matches exactly what the method was designed to do. We know where the rough edges are, no working car detection, no appearance based tracker yet, and only one sequence properly evaluated for tracking, and each of those has a clear next step attached to it rather than being an open question.

References

- [1] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2016.
- [2] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2017.
- [3] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multiple object tracking by associating every detection box," in Proc. Eur. Conf. Comput. Vis. (ECCV), 2022.
- [4] A. Milan, L. Leal Taixe, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multiple object tracking," arXiv preprint arXiv:1603.00831, 2016.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "YOLOv8," Ultralytics, 2023. Available online at <https://github.com/ultralytics/ultralytics>.

Author Contributions

Data preparation, downloading MOT16, verifying annotations, and writing the preprocessing scripts in Section 3, was led by M.R.A. Rahman and M.M.M.A. Nular. Detector training and tracker integration, the YOLOv8n configuration and the SORT and ByteTrack pipelines in Section 4, was led by A. Piraveen and V. Thanush. The evaluation script, the results analysis in Section 6, and this report were put together jointly by all four members, who also share responsibility for the project repository.